

how do i use an api key and connect it to my colab

Yep! If you're using **Google Colab**, the cleanest way is to store your API key in **Colab's Secrets** rather than typing the key directly into your notebook. This keeps the key out of your code, which is also what Groq recommends.

I'll show you the process using **Groq**, since that's a common choice for an LLM API.

1. Get your API key

Go to [Groq Console](#) and create an API key.

You'll get something that looks roughly like:

gsk_XXXXXXXXXXXXXXXXXXXXXX

Don't send the key to me or paste it into your notebook code. 🗝️

2. Put the key into Colab Secrets

In your Colab notebook:

1. Look at the **left sidebar**.
2. Click the  **key icon** called **Secrets**.
3. Click **Add new secret**.
4. For the **Name**, enter:

GROQ_API_KEY

5. For the **Value**, paste your actual Groq API key.
6. Turn on **Notebook access** for that secret.

So you'll have:

Name: GROQ_API_KEY

Value: gsk_.....

3. Connect the secret to Python

In a new Colab cell, run:

```
from google.colab import userdata
```

```
GROQ_API_KEY = userdata.get("GROQ_API_KEY")
```

```
print("API key loaded:", GROQ_API_KEY is not None)
```

You should get:

API key loaded: True

Notice that we're **not printing the actual key**. That's important.

4. Install the Groq library

Run:

```
!pip install -q groq
```

Groq's official Python library is installed with `pip install groq`.

5. Create your Groq client

Then:

```
from groq import Groq
```

```
from google.colab import userdata
```

```
client = Groq(  
    api_key=userdata.get("GROQ_API_KEY")  
)
```

Now your Colab notebook is connected to Groq. 

6. Test that it works

Try:

```
response = client.chat.completions.create(  
    model="llama-3.3-70b-versatile",  
    messages=[  
        {  
            "role": "user",  
            "content": "Say hello!"  
        }  
    ]  
)
```

```
print(response.choices[0].message.content)
```

If everything is working, you'll get a response from the model.

Groq's API uses the `chat.completions.create()` pattern shown in its official documentation.

The whole setup in Colab

You can think of it as:

Your Groq API key



Colab Secrets



```
userdata.get("GROQ_API_KEY")
```



Groq client



LLM



Response

And **you only need to set the secret once per notebook/account setup**, rather than putting something like this in your code:

```
api_key = "gsk_..."
```

Don't do that, especially if you're going to push the notebook to GitHub. 🔒 Groq specifically recommends environment variables or secret-management systems instead of hardcoding keys.

If you're trying to connect **a different API** such as Gemini or Hugging Face rather than Groq, tell me which one and I can give you the exact Colab setup.

JSON is basically a **standard way of packaging data so that different programs can easily understand and exchange it**. 📦

JSON stands for **JavaScript Object Notation**, but you don't need to know JavaScript to use it.

Think of JSON as a structured form

Suppose you have this information about a student:

Name: Fredda

Age: 21

Course: Data Science

You could represent it as JSON:

```
{  
  "name": "Fredda",  
  "age": 21,  
  "course": "Data Science"  
}
```

The important thing is that each piece of information has a **key** and a **value**:

"name" → "Fredda"

"age" → 21

"course" → "Data Science"

Why is JSON useful with APIs?

This is where it becomes especially important for your LLM work.

When your Colab sends an API request, it often sends the information as JSON:

```
{  
  "model": "llama-3.3-70b-versatile",  
  "temperature": 0,  
  "messages": [  
    {  
      "role": "user",  
      "content": "Summarize this loan application."  
    }  
  ]  
}
```

```
}
```

The API receives that structured data and knows:

- **which model** to use
- **what temperature** to use
- **what message** you want the model to process

Then the API sends a response back, which is also structured data.

For example:

```
{  
  "response": "The applicant is requesting a loan of GHS 5,000..."  
}
```

Your Python program can then grab the specific part it needs.

JSON is especially useful for your extraction task

Suppose your loan application says:

"I am requesting GHS 5,000 to expand my food business. My monthly profit is approximately GHS 1,200."

Instead of asking the LLM to give you a paragraph, you can ask it to produce:

```
{  
  "loan_amount": 5000,  
  "purpose": "Expand food business",  
  "monthly_profit": 1200  
}
```

Now another program can easily use that information.

For example:

```
data["loan_amount"]
```

would give:

```
5000
```

And:

```
data["monthly_profit"]
```

would give:

1200

That's much easier for a computer to work with than trying to interpret a paragraph.

JSON vs normal text

Imagine you're giving information to another human:

The applicant wants 5000 cedis to expand their food business
and makes about 1200 cedis per month.

A human can understand that.

But if you're giving the information to a **database or another program**, this is much better:

```
{  
  "loan_amount": 5000,  
  "purpose": "Expand food business",  
  "monthly_profit": 1200  
}
```

The second version is **structured**.

That's the big idea:

JSON turns information into a predictable structure that software can easily read, store, send, and process.

In your LLM/API setup

The whole pipeline can look like:

Loan application



LLM



JSON



Python / Database



Decision-support system

So when your lab asks you to **extract structured data as JSON**, it's basically asking you to turn messy human language into a neat data package that the rest of the system can actually work with. 📦 ✨

if im using colab and i have different prompts i want to commit in my repo, how do i get the commit hash

Yep. If you're using **Google Colab + GitHub**, each time you commit your notebook, GitHub gives that commit a **commit hash**. The hash identifies that exact version of your work.

1. Make your changes in Colab

For example, you might have different prompts:

```
PROMPT_V1 = "Summarize this loan application."
```

```
PROMPT_V2 = """Summarize this loan application.
```

```
Include the requested loan amount, purpose, and repayment information."""
```

Once you've added/tested the version you want to save, commit it.

2. Commit from Colab

If you're using Colab's GitHub integration:

File → Save a copy in GitHub

Then choose:

- Repository
- Branch
- File path

Add a commit message, such as:

Add summary prompt V2

Then click **OK**.

That creates a Git commit.

3. Get the commit hash from GitHub

Go to your GitHub repository and:

1. Click **Commits**.
2. Find your commit, e.g.

Add summary prompt V2

You'll see something like:

a7f3c92

That's the **short commit hash**.

If you click the commit, GitHub will show the full hash, something like:

a7f3c92e8b1f4d6c2a9...

Usually, the short version such as a7f3c92 is enough when your lab asks for a commit hash.

 If you have multiple prompt versions

This is actually a good way to organize your lab.

For example:

Version	What you changed	Commit
Prompt V1	Basic summarization	a7f3c92
Prompt V2	Added specific fields	b81d4e1
Prompt V3	Improved factuality instructions	c52ab77

Then if your assignment asks:

"Provide the commit hash for your prompt engineering changes"

you can point to the commit where that particular prompt was introduced.

Even easier: use Git in Colab's terminal

If your Colab environment has the repository cloned, you can run:

```
!git log --oneline
```

You'll get something like:

c52ab77 Improve decision-support prompt

b81d4e1 Add structured extraction prompt

a7f3c92 Add summary prompt V2

The first 7 characters are the short commit hashes. 🎯

And if you want the hash of the **current commit specifically**:

```
!git rev-parse HEAD
```

That gives you the full hash.

One thing to watch: if you're editing the notebook in Colab and then committing through GitHub's web integration, make sure the notebook has actually been saved/committed before grabbing the hash. Otherwise you may accidentally record the hash of an older version.

Today 10:35 PM

what advantage does an LLM have over a model I've built myself and vice versa

It depends on **what you're trying to solve**. An LLM and a model you build yourself are really two different tools.

For example, imagine you need a system that reads loan application letters and helps a loan officer make decisions.

🤖 LLM vs. your own ML model

	LLM (e.g. Groq + Llama)	Model you build yourself
Training	Already pretrained on huge amounts of data	You train it yourself
Setup	Relatively quick	Takes more time
Understands text	★★★★★	Depends on your model
Customization	Prompt/fine-tuning	Full control
Data needed	Can work with very little task-specific data	Usually needs training data
Predictability	Can sometimes produce unexpected outputs	Usually more controlled
Cost	API usage can cost money	Training/inference can be cheaper once deployed

	LLM (e.g. Groq + Llama)	Model you build yourself
Transparency	Harder to understand exactly why it produced something	You have much more control over how it works
Specialized task	Can be very capable immediately	Can outperform an LLM if designed/trained specifically
Maintenance	Provider handles the model	You maintain it

Where an LLM has the advantage

1. It can understand messy human language

Suppose you receive:

"I'm hoping to borrow about five thousand cedis so I can buy another freezer and increase the amount of drinks I sell at my shop."

An LLM can infer:

```
{
  "amount": 5000,
  "purpose": "Purchase another freezer to expand drink sales"
}
```

You don't have to explicitly program every possible way someone could say **"I need GHS 5,000."**

That's one of the biggest strengths of LLMs.

2. You don't need thousands of examples to get started

With a traditional supervised ML model, you might need something like:

10,000 applications

↓

label them

↓

train model

↓

evaluate



tune



deploy

With an LLM, you can often give it instructions and examples directly:

Extract:

- applicant name
- loan amount
- purpose
- monthly income
- collateral

Then give it the application.

That's called **prompt engineering**.

3. One LLM can perform many tasks

The same model could:

Application



Summarize



Extract information



Classify risk factors



Explain the reasoning



Generate a decision-support brief

You don't necessarily need a completely separate model for each task.

Where your own model has the advantage

Here's where building your own model gets interesting.

1. You have much more control

Suppose you build a model specifically for loan risk.

You decide:

Inputs:

- income
- loan amount
- repayment history
- existing debt
- business age

Output:

- probability of default

The model isn't deciding what information matters based on a general language understanding. **You define the inputs and target.**

That can make the system much more predictable.

2. It can be cheaper at scale

Imagine you have **10 million loan applications**.

Sending all 10 million through an external LLM API could become expensive.

A small model that you've trained and deployed yourself might process those applications much more cheaply.

So:

Small specialized model



Millions of predictions



Potentially low cost per prediction

3. You can specialize it extremely well

Suppose you have 500,000 historical loan applications from your specific institution.

You could train a model specifically on that data.

The model might learn patterns such as:

income

loan amount

repayment history

business type



default probability

A general LLM doesn't automatically have access to your private historical data.

4. Privacy can be easier to control

With your own model, sensitive information can potentially remain entirely inside your organization's infrastructure.

With an external API, you're sending data to an external provider, so you have to consider the provider's data-handling policies, security, and contractual requirements.

So which one is "better"?

Neither universally.

It's more like:

Use an LLM when:

- The input is **unstructured text**
- You need natural-language understanding

- You don't have a huge labeled dataset
- You need to build something quickly
- You need multiple language-based tasks
- You want flexible behavior

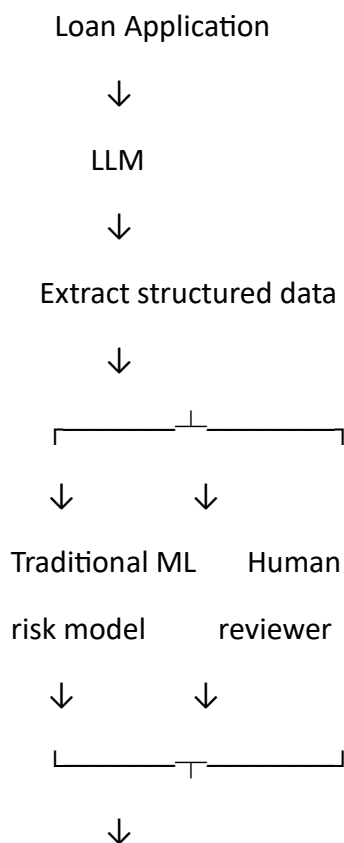
Build your own model when:

- You have lots of relevant training data
- The task is narrowly defined
- You need highly predictable outputs
- You need low-cost inference at huge scale
- You need maximum control
- You need a specialized model

And there's actually a third option 🧐

You don't necessarily have to choose one.

You can combine them:



Decision support

For example, the **LLM** could read:

"I've been running my provisions shop for three years..."

and turn that into structured features:

business_age = 3

loan_amount = 5000

purpose = business expansion

Then your **own ML model** could take those numerical features and calculate:

default_probability = 0.17

And finally, a **human loan officer** makes the actual decision.

That hybrid approach can be much more powerful than treating an LLM as a magical replacement for every other model. 🧩

In one sentence: an LLM gives you **flexibility and language understanding**, while a model you build yourself gives you **control, specialization, and predictability**.